1. The Core Dev Loop: Full-Stack Feature Development & Testing

Objective: Orchestrate a complete pipeline from specification to a tested, peer-reviewed codebase.

> **[TASK]:** Implement a new full-stack feature starting from initial specification through implementation, QA testing, and automated code review.
>
> **[TOOLS]:** Use spec-driven-workflow, senior-fullstack, senior-qa, and code-reviewer.
>
> **[DETAILS]:** The feature to build is [FEATURE]. The target frontend technology is [FRONTEND] and the backend is [BACKEND].
>
> **[OTHER]:** Execute sequentially: 1. Generate the technical spec. 2. Write the frontend components and backend API routes. 3. Generate unit and E2E tests covering both happy and unhappy paths. 4. Run a hostile code review for complexity and SOLID violations.
>
> **[OPTIONAL]:** Once complete, output a structured summary of all code changes, test coverage deltas, and file structures to be saved into the project's internal memory.

---

2. The DevOps & Security Loop: Secure Infrastructure Provisioning

Objective: Design cloud architecture, write the infrastructure code, scan it for vulnerabilities, and build the deployment pipeline.

> **[TASK]:** Design, provision, and secure cloud infrastructure along with an automated deployment pipeline.
>
> **[TOOLS]:** Use [CLOUD_TOOL], terraform-patterns, cloud-security, and ci-cd-pipeline-builder.
>
> **[DETAILS]:** The target architecture requires [INFRASTRUCTURE].
>
> **[OTHER]:** Execute sequentially: 1. Output the architectural diagram and service map. 2. Generate the Terraform modules utilizing state management best practices. 3. Scan the generated IaC against MITRE ATT&CK cloud matrices. 4. Output the YAML pipeline configuration for automated deployment.
>
> **[OPTIONAL]:** Output the final environment variables schema needed for env-secrets-manager and record the architecture decisions as an ADR for internal memory.

---

3. The Backend Data Loop: Schema Migration & Secure API Generation

Objective: Safely transition database schemas, map them to an ORM, expose them via APIs, and guarantee test coverage.

> **[TASK]:** Execute a database schema migration, implement the associated backend API routes, and generate complete test suites.
>
> **[TOOLS]:** Use database-designer, sql-database-assistant, senior-backend, and api-test-suite-builder.
>
> **[DETAILS]:** The required data structural change is [MIGRATION]. The database engine is [DATABASE] and the ORM in use is [ORM].
>
> **[OTHER]:** Execute sequentially: 1. Design the normalized schema and output the SQL migration file. 2. Update the ORM models. 3. Write the backend API controllers with strict input validation. 4. Generate test suites for every newly created route.
>
> **[OPTIONAL]:** Run senior-security to check the new routes for OWASP Top 10 vulnerabilities before outputting the final git commit state for internal memory.

---

4. The AI/ML Integration Loop: Secure Pipeline Deployment

Objective: Scaffold AI features, design agent communication, optimize for token limits, and harden against prompt injections.

> **[TASK]:** Architect, implement, and secure a multi-agent or retrieval-augmented generation (RAG) pipeline.
>
> **[TOOLS]:** Use agent-designer, rag-architect, ai-security, and llm-cost-optimizer.
>
> **[DETAILS]:** The system's primary capability must be [CAPABILITY]. The targeted embedding and generation models are [MODELS].
>
> **[OTHER]:** Execute sequentially: 1. Draft the agent workflow orchestration. 2. Write the code for chunking, embedding, and retrieval. 3. Apply prompt caching and routing logic to minimize token spend. 4. Implement adversarial robustness filters against MITRE ATLAS threats.
>
> **[OPTIONAL]:** Output a complete README block detailing the agent protocols and cost-optimization thresholds for team onboarding and internal memory.

---

5. The UI/UX Quality Loop: Frontend Refactor & Accessibility Audit

Objective: Build visually robust components, guarantee accessibility compliance, and automate user interaction testing.

[TASK]: Develop or refactor a highly optimized frontend interface with integrated accessibility auditing and E2E testing.

[TOOLS]: Use senior-frontend, a11y-audit, and playwright-pro.

[DETAILS]: The UI component to build/refactor is [COMPONENT]. The target frontend framework and styling libraries are [FRAMEWORK].

[OTHER]: Execute sequentially: 1. Scaffold the component with strict typing interfaces. 2. Run the accessibility audit to ensure screen reader compatibility and correct ARIA states. 3. Generate the automated testing templates for user interactions (clicks, keyboard navigation).

[OPTIONAL]: If DOM dependency issues or visual bugs are anticipated, trigger focused-fix to systematically repair them before outputting the fully audited component state for internal memory.

Here are the foundational prompt templates focused specifically on Quality Assurance (QA), DevOps, and Cloud workflows. These follow the exact same modular structure and use **single-word global variables** for seamless integration into your broader automation pipeline.

6. The End-to-End QA Loop: Test Automation & Validation

Objective: Establish a comprehensive, automated testing suite for an existing feature, covering unit, integration, end-to-end (E2E) flows, and code quality.

> **[TASK]:** Generate and configure a complete automated test suite for an existing feature, ensuring maximum coverage and reliability.
>
> **[TOOLS]:** Use senior-qa, playwright-pro, tdd-guide, and code-reviewer.
>
> **[DETAILS]:** The target feature requiring test coverage is [FEATURE]. The core testing framework to utilize is [TESTRUNNER].
>
> **[OTHER]:** Execute sequentially: 1. Map out the critical happy and unhappy user paths. 2. Generate unit and integration tests using standard Red-Green-Refactor principles. 3. Build resilient E2E browser tests to validate full system functionality. 4. Run an automated review to ensure the new tests are not flaky and do not contain code smells.
>
> **[OPTIONAL]:** Output a comprehensive Istanbul coverage report and document any edge cases that require manual validation for internal memory.

---

7. The CI/CD Pipeline Loop: Automated Build & Deployment

Objective: Construct a highly secure, automated pipeline that builds, tests, and deploys code across different environments.

> **[TASK]:** Build a secure, multi-stage Continuous Integration and Continuous Deployment (CI/CD) pipeline.
>
> **[TOOLS]:** Use ci-cd-pipeline-builder, senior-devops, dependency-auditor, and env-secrets-manager.
>
> **[DETAILS]:** The codebase resides in [REPOSITORY] and needs to be deployed to the [ENVIRONMENT] environment.
>
> **[OTHER]:** Execute sequentially: 1. Audit current dependencies for obsolescence or vulnerabilities before pipeline execution. 2. Auto-generate the pipeline YAML configuration with stages for build, test, and deploy. 3. Integrate strict secret scanning and environment variable management. 4. Configure deployment triggers and rollback mechanisms.
>
> **[OPTIONAL]:** Integrate automated alerting for pipeline failures and output the final deployment sequence documentation for internal memory.

---

8. The Cloud Migration Loop: Architecture Modernization

Objective: Plan, containerize, and execute the migration of a legacy system to a modern cloud-native architecture.

> **[TASK]:** Architect and execute a phased migration from a legacy system to a containerized, cloud-native infrastructure.
>
> **[TOOLS]:** Use migration-architect, [CLOUD_TOOL], docker-development, and observability-designer.
>
> **[DETAILS]:** The current infrastructure is [LEGACY_SYSTEM] and the goal is to migrate to a modern [ARCHITECTURE].
>
> **[OTHER]:** Execute sequentially: 1. Generate a phase-by-phase migration architecture plan to ensure zero downtime. 2. Optimize and harden the Dockerfiles for multi-stage, secure builds. 3. Provision the required serverless/container orchestration infrastructure using the designated cloud tool. 4. Design the observability stack to monitor the new deployment for performance regressions.
>
> **[OPTIONAL]:** Output a complete rollback runbook and a performance profiling baseline to compare the new architecture against the legacy system for internal memory.

---

9. The Cloud Security Hardening Loop: Threat Mitigation

Objective: Audit existing cloud infrastructure, identify vulnerabilities, and apply strict access controls and remediation.

> **[TASK]:** Conduct a comprehensive security audit of existing cloud infrastructure and implement automated remediation and secret management.
>
> **[TOOLS]:** Use cloud-security, secrets-vault-manager, threat-detection, and incident-response.
>
> **[DETAILS]:** The target infrastructure to audit is [INFRASTRUCTURE]. The required regulatory or internal standard is [COMPLIANCE].
>
> **[OTHER]:** Execute sequentially: 1. Scan the cloud environment for IAM misconfigurations, exposed buckets, and IaC gaps. 2. Transition all hardcoded credentials or loose variables into a secure vault system. 3. Implement anomaly detection and threat hunting sweeps. 4. Draft an updated SEV1-4 classification and incident escalation runbook based on the newly applied controls.

[OPTIONAL]: Generate a mapping of the mitigated risks against the MITRE ATT&CK framework and output an executive summary of the fortified posture for internal memory.

---

10. The Agile Infrastructure Loop: TDD & Spec-First Provisioning

Objective: Drive the creation of a new microservice or module purely through Test-Driven Development and strict specification before writing operational code.

[TASK]: Initialize and build a new system module using strict Test-Driven Development (TDD) and adversarial review.

[TOOLS]: Use spec-driven-workflow, tdd-guide, adversarial-reviewer, and senior-backend.

[DETAILS]: The new module to be built is [MODULE]. The core business logic requirement is [REQUIREMENT].

[OTHER]: Execute sequentially: 1. Write the strict technical specifications and API contracts before any code is authored. 2. Generate failing test cases that reflect the business requirements. 3. Implement the backend logic to turn the tests green. 4. Deploy an adversarial reviewer persona to actively try and break the implementation or find logical blind spots.

[OPTIONAL]: If the adversarial review finds flaws, trigger a focused fix loop until tests pass, then output the finalized spec-to-code trace for internal memory.

Here are the bridge templates designed to seamlessly connect your foundational engineering work (like feature development) directly into Git workflows, automated CI/CD checks, and cloud staging. These continue the use of single-word global variables for easy programmatic swapping across your repository management and release cycles.

11. The Pull Request Gatekeeper: Git Review & Release Prep

Objective: Bridge a completed feature branch into the main branch by enforcing strict peer review, isolating the code for testing, and auto-generating the release notes.

[TASK]: Execute a comprehensive Pull Request review, audit for security regressions, and generate structured release documentation.

[TOOLS]: Use git-worktree-manager, pr-review-expert, skill-security-auditor, and changelog-generator.

[DETAILS]: The active pull request is [PULLREQUEST] merging from [BRANCH] into the main integration branch.

[OTHER]: Execute sequentially: 1. Isolate the PR branch using git worktrees to prevent local state corruption. 2. Run a blast radius and coverage delta analysis on the proposed code. 3. Audit any new dependencies or AI skills for malicious code. 4. Parse the conventional commits to generate the structured release notes.

[OPTIONAL]: Output the final PR approval summary and append the generated changelog to the corresponding Jira ticket using jira-expert for internal memory.

---

12. The Monorepo Pipeline Integrator: Repo-to-Cloud CI/CD

Objective: Map dependencies within a complex multi-package repository and build an optimized, targeted CI/CD pipeline that only builds what has changed.

[TASK]: Analyze a monorepo structure to update dependencies and generate a hyper-targeted CI/CD pipeline configuration.

[TOOLS]: Use monorepo-navigator, dependency-auditor, ci-cd-pipeline-builder, and release-manager.

[DETAILS]: The target repository is [REPOSITORY] and the specific updated package/workspace is [PACKAGE].

[OTHER]: Execute sequentially: 1. Map the internal dependencies of the updated package against the rest of the workspace. 2. Audit the package for obsolete or vulnerable external dependencies. 3. Auto-generate the GitHub Actions/GitLab CI pipeline configuration to trigger builds *only* for the affected package and its dependents. 4. Coordinate the deployment versioning strategy.

[OPTIONAL]: Output a graph representation of the affected deployment packages and the final YAML pipeline file for internal memory.

---

13. The Staging Bridge: Feature Branch to Cloud Preview

Objective: Take a newly validated feature branch, containerize it, and deploy it to an ephemeral cloud preview environment for stakeholder testing.

[TASK]: Containerize a validated feature branch and provision a temporary cloud preview environment for staging.

[TOOLS]: Use docker-development, helm-chart-builder, [CLOUD_TOOL], and team-communications.

[DETAILS]: The tested feature branch is [BRANCH] and the target deployment cluster is [CLUSTER].

[OTHER]: Execute sequentially: 1. Generate or optimize the Dockerfile using multi-stage builds for a minimal footprint. 2. Scaffold the Helm chart values and RBAC policies required for the specific cluster. 3. Provision the ephemeral staging resources using the designated cloud architect tool. 4. Draft a broadcast message with the preview URL and testing instructions.

[OPTIONAL]: Integrate a basic performance-profiler run against the preview URL to establish a baseline before notifying the team, outputting the metrics for internal memory.

---

14. The SecOps Hotfix Bridge: Vulnerability to Deployed Patch

Objective: Connect an active security incident or penetration testing finding directly to a systematic codebase fix, IaC update, and automated deployment.

[TASK]: Triage a detected security vulnerability, systematically apply a codebase and infrastructure fix, and push the hotfix through the CI/CD pipeline.

[TOOLS]: Use incident-response, focused-fix, terraform-patterns, and jira-expert.

[DETAILS]: The active vulnerability classification is [VULNERABILITY] tracked under incident ticket [INCIDENT].

[OTHER]: Execute sequentially: 1. Classify the severity and outline the forensic evidence of the vulnerability. 2. Execute a systematic deep-dive repair across the affected files and dependencies to patch the exploit. 3. Update the required Sentinel/OPA policies and state management via Terraform to prevent infrastructure-level recurrence. 4. Auto-update the tracking ticket with the resolution state and pipeline trigger confirmation.

[OPTIONAL]: Trigger a post-mortem review utilizing adversarial-reviewer against the applied patch to ensure the fix cannot be bypassed, logging the results for internal memory.

Here are the foundational security templates focused on Red Teaming (offensive), Blue Teaming (defensive), and their intersection. These templates continue utilizing the **single-word global variables** for programmatic injection across your security workflows.

15. The Red Team Campaign: Offensive Attack Simulation

Objective: Execute an authorized, structured attack simulation to test organizational defenses, identify exploit paths, and assess the security of critical assets.

[TASK]: Execute an authorized red team campaign to simulate an advanced persistent threat (APT) and identify exploit paths.

[TOOLS]: Use red-team, security-pen-testing, and cloud-security.

[DETAILS]: The target scope is [SCOPE] and the designated crown jewels are [ASSETS].

[OTHER]: Execute sequentially: 1. Develop an ATT&CK kill-chain plan targeting the crown jewels without triggering alarms. 2. Conduct authorized API and network penetration testing focusing on OWASP vulnerabilities. 3. Assess cloud environments for IAM misconfigurations, lateral movement paths, and S3 exposures. 4. Document all successful exploits, bypassed controls, and privilege escalations.

[OPTIONAL]: Output a comprehensive attack narrative and a prioritized list of compromised assets for internal memory.

---

16. The Blue Team Threat Hunt: Defensive Sweeps & Detection

Objective: Proactively search through network and system telemetry to identify anomalous behaviors, hidden threats, and active indicators of compromise.

[TASK]: Conduct a proactive threat hunting sweep to identify anomalous behaviors and active indicators of compromise (IOCs).

[TOOLS]: Use threat-detection, senior-secops, and incident-response.

[DETAILS]: The environment to monitor is [ENVIRONMENT] and the relevant threat intelligence feed or framework is [FRAMEWORK].

[OTHER]: Execute sequentially: 1. Ingest logs and perform z-score analysis to identify baseline deviations and anomalous user behavior. 2. Sweep the environment for known IOCs associated with recent threat actor campaigns. 3. Run DAST/SAST scans on critical applications within the perimeter to find unpatched CVEs. 4. Draft an incident response escalation path for any confirmed positive findings using NIST 800-61 protocols.

[OPTIONAL]: Output a threat hunt summary report detailing the swept systems, baseline anomalies, and false positive tuning recommendations for internal memory.

---

17. The Purple Team Exercise: Collaborative Attack & Defense

Objective: Bridge the gap between offensive testing and defensive monitoring by running simultaneous attacks and detection validations to immediately patch visibility gaps.

> **[TASK]:** Facilitate a collaborative Purple Team exercise bridging offensive simulation with immediate defensive detection and remediation.
>
> **[TOOLS]:** Use red-team, threat-detection, incident-commander, and jira-expert.
>
> **[DETAILS]:** The targeted system for this exercise is [SYSTEM] and the specific attack vector being tested is [VECTOR].
>
> **[OTHER]:** Execute sequentially: 1. The Red Team agent executes the designated attack vector against the target system. 2. The Blue Team agent simultaneously monitors security telemetry to verify if the attack was successfully detected and alerted. 3. Initiate the incident commander protocol to simulate triage and containment of the "breach". 4. Generate Jira tickets detailing the logging/visibility gaps and missing preventative controls.
>
> **[OPTIONAL]:** Output a synchronized timeline showing the exact moment of exploitation versus the time of detection for internal memory.

---

18. The Deep Application Pen-Test: Focused AppSec & Secrets

Objective: Perform an intensive, authorized penetration test on a specific application boundary to uncover code vulnerabilities, API flaws, and leaked credentials.

> **[TASK]:** Perform an authorized, deep-dive penetration test on a specific application boundary to uncover API and secret vulnerabilities.
>
> **[TOOLS]:** Use security-pen-testing, adversarial-reviewer, env-secrets-manager, and focused-fix.
>
> **[DETAILS]:** The target application repository is [REPOSITORY] and the primary authentication mechanism is [AUTH].
>
> **[OTHER]:** Execute sequentially: 1. Conduct hostile code review and dynamic API testing mapping directly to the OWASP Top 10. 2. Scan the codebase and commit history for leaked secrets, hardcoded credentials, or exposed API keys. 3. Formulate systematic deep-dive repair plans for any discovered vulnerabilities. 4. Push any exposed secrets to the vault manager for immediate rotation and apply the codebase fixes.
>
> **[OPTIONAL]:** Output the final vulnerability remediation report and the Git diff of the applied hotfixes for internal memory.

---

19. The AI Adversarial Stress Test: LLM Security & Red Teaming

Objective: Stress-test AI models, RAG pipelines, and agentic workflows against prompt injection, jailbreaks, and unauthorized data exfiltration.

> **[TASK]:** Stress-test an AI system against prompt injection, data exfiltration, and adversarial jailbreaks.
>
> **[TOOLS]:** Use ai-security, adversarial-reviewer, rag-architect, and senior-ml-engineer.
>
> **[DETAILS]:** The target AI model or pipeline is [PIPELINE] and the specific threat matrix being evaluated is [MATRIX] (e.g., MITRE ATLAS).
>
> **[OTHER]:** Execute sequentially: 1. Deploy adversarial personas to attempt jailbreaks, prompt injections, and system prompt extraction. 2. Evaluate the RAG pipeline for unauthorized context retrieval, authorization bypasses, or data leakage. 3. Implement defensive guardrails, input sanitization, and robustness filters based on the successful attacks. 4. Monitor the model deployment for input drift or repeated hostile patterns.
>
> **[OPTIONAL]:** Output a security resilience score and a comprehensive log of all successful adversarial payloads for internal memory.

Here are the foundational AI & Data templates, incorporating both the advanced agent orchestration skills and deep machine learning/data engineering tools. They utilize the same **single-word global variables** for programmatic integration into your SOP framework.

20. The Data Engineering Pipeline Loop: DataOps & ETL

Objective: Architect, build, and deploy a highly reliable, scalable data pipeline to prepare raw data for machine learning or enterprise analytics.

> **[TASK]:** Design and implement a robust data pipeline including extraction, transformation, and load (ETL) operations with DataOps monitoring.
>
> **[TOOLS]:** Use senior-data-engineer, performance-profiler, and ci-cd-pipeline-builder.
>
> **[DETAILS]:** The source data environment is [SOURCE] and the target data warehouse or analytical engine is [DESTINATION].
>
> **[OTHER]:** Execute sequentially: 1. Architect the ETL/ELT pipeline utilizing appropriate distributed processing frameworks (e.g., Spark, Airflow, dbt). 2. Write the transformation logic to clean, normalize, and format the data. 3. Implement

DataOps checks for data quality, schema evolution, and anomaly detection. 4. Configure batch or real-time ingestion streams (e.g., Kafka).

[OPTIONAL]: Output a comprehensive data lineage map and the pipeline deployment YAML configuration for internal memory.
